DISPATCH mechanism — Flow of the journey analysis waves

SKILL.md -> reads 25_dispatch-orchestration.md

Platform detection:
- Claude Code -> isolated sub-agents (parallel)
- Others -> sequential inline execution

Wave 0 — Location (main context)

Read args: directory or direct files

Identify .har + Coverage-*.json + cwv.json (optional)

Check HAR existence + size
(selective reading if > 5 MB)

✅Wave 0 done — files ready
The following dispatches read the disk files.

Wave 1 — HAR + Coverage + CWV + env-data + Security (5 sub-agents in parallel)

Dispatch analyze-har
(20_har-analysis.md)
python3 analyze_har.py <source-dir>
Traffic + performance (requests, domains,
HTTP codes, duplicates, blocking resources)
-> har-analysis.json

Dispatch analyze-coverage
(30_coverage-analysis.md)
Format A or B detected
JS/CSS metrics per page
(total KB, unused KB, %)
Top 5 unused JS + Top 3 unused CSS
Multi-page summary + outliers
-> coverage-analysis.json

Dispatch collect-cwv
(37_lighthouse.md)
collect_cwv_pagespeed.py --strategy both
PageSpeed Insights + CrUX (mobile+desktop,
accessibility/best-practices included from the
start) ; automatic Lighthouse fallback if
GOOGLE_API_KEY absent or 0 usable metric
-> cwv.json

Dispatch collect-env-data
(45_efootprint.md, Step 20)
collect_env_data.py <source-dir>
HAR + CrUX (device mix) + ipinfo.io
(country/provider) + internal SimilarWeb API
(country mix + traffic)
-> env-data.json

Dispatch extractor-security-headers
(39_eof-audit.md)
analyze_security_headers.py <source-dir>
Local security score from the headers
already in the .har, no network call
-> security-headers-analysis.json

Independence verified in code, not assumed:
analyze-har and extractor-security-headers ALSO
read env-data.json, but in SOFT degradation
(file absent -> less precise classification /
page filter ignored, never a crash).
Success condition:
har-analysis.json AND coverage-analysis.json
mandatory (blocking warning if absent).
cwv.json / env-data.json / security-headers-analysis.json
best-effort (absence signaled, never blocking).

Wave 2 — EOF extractors depending on env-data.json (2 sub-agents in parallel)

Dispatch extractor-html-css
depends_on="collect-env-data"
parse_html_criteria.py <source-dir>
Live re-fetch of already audited pages
(the .har does not contain the HTML/CSS bodies)
Archived under html-pages/ (job 27)
-> html-css-criteria.json

Dispatch extractor-wellknown
depends_on="collect-env-data"
scan_wellknown.py <source-dir>
Direct fetch security.txt / robots.txt /
sitemap.xml on the domain deduced from env-data.json
-> wellknown-scan.json

HARD DEPENDENCY (not a degradation), verified
line by line in the code:
parse_html_criteria.py and scan_wellknown.py
sys.exit(1) if env-data.json is absent or empty.
Never dispatch this wave before having
confirmed env-data.json on disk (Wave 1 done) —
a simple retry does not compensate for this hard dependency.

Merge + rest (main context)

Read har-analysis.json + coverage-analysis.json
(mandatory) + cwv.json / env-data.json /
security-headers-analysis.json / html-css-criteria.json /
wellknown-scan.json (best-effort, Wave 1/2)

Step 35 — har_metrics.py
Score/grade A-G per page calculation
(official cnumr/ecoindex formula)

Limits:
DOM=0 if HAR without HTML body
SPA: DOM = initial state only
-> documented in the report
No OUTPUT_FILE reused elsewhere:
free console validation, outside dispatch

If cwv.json absent (Wave 1 failed):
report without CWV, HAR onLoad used
as LCP proxy

Step 39 — run_eof.py <source-dir>
Confronts har-analysis.json / coverage-analysis.json
(mandatory) + cwv.json / env-data.json /
security-headers-analysis.json / html-css-criteria.json /
wellknown-scan.json (best-effort, already produced by
Wave 1/2 — no more annex extractors to launch here)
against eof-reference.json (54 criteria + 16 quick diagnostic)
-> eof-audit-results.json, eof-filled.md,
eof-radar-<domain>.svg

STOP HERE, nothing is automatic after:
generate_questionnaire.py / parse_questionnaire.py /
merge_lots.py are manual steps taken on purpose
(mission decision), never triggered by this skill.
Trap: never rerun run_eof.py alone after a
questionnaire read-back (it silently overwrites the answers,
it never re-reads lots/questionnaire-review.json).
Always parse_questionnaire.py -> merge_lots.py
-> generate_report_html.py.

Step 40 — generate_report_html.py
Sections: Recommendations P1/P2/P3,
GreenIT best practices, CWV (if cwv.json),
e-footprint CO2e (if efootprint-python-summary.json),
EOF optimization potential (if eof-audit-results.json,
measured + questionnaire merged),
Appendices (EcoIndex, Traffic, Dead code)
-> journey-report-YYYY-MM-DD.html
(internal check_report_values.py, non-blocking)

deliver the report path to the user

Conditional CO2e section:
displayed only if efootprint-python-summary.json
is present in the audit-dir

Optional step - e-footprint CO2e (skill /efootprint)

User launches /efootprint ?

yes

E1 - collect_env_data.py
HAR extraction (weight, duration, IP)
API ipinfo.io (country + provider)
CrUX (desktop/mobile device mix)
-> env-data.json

Cache mode:
If env-data.json already exists (often already
produced by the Wave 1 of /analyse-parcours,
collect-env-data block), skip API calls
(except --refresh)

E2 - AskUserQuestion
- Annual traffic (visits/year) [REQUIRED]
- Cloud instance (if AWS/Azure/GCP detected)

E3 - run_efootprint.py
Boavizta model (devices/network/server/storage)
Manufacturing + energy calculation
-> efootprint-boavizta-model.json
-> efootprint-python-summary.json

Results:
- Total CO2e (kg/year)
- Per visit (g CO2e/visit)
- Manufacturing/energy breakdown
- Hypotheses + confidence badges

E4 - Regenerate HTML report
generate_report_html.py
-> report enriched with the CO2e section

CO2e impact summary
delivered to the user

no

Report without CO2e section